

samedi 7 mars 2026 00:30

Ça créer des allocations privées ce qui est très monitoré par les solution de sécurité. Donc le but ici est de changer ces fonctions winAPI par la fonction

0x7f656491000	Image: Commit	12416	RX	C:\Windows\System32\runtime4...	2416	2416
0x7f656381000	Image: Commit	1,372,168	RX	C:\Windows\System32\wbased...	33616	33616
0x7f656461000	Image: Commit	3616	RX	C:\Users\User\Desktop\LocalThread...	2816	2816
0x7f65620000	Private: Commit	416	RWX		416	416
0x7f6580000	Private: Commit	1216	RW+G	Stack (thread 20744)		
0x7f65636000	Private: Commit					
0x7f656320000	Image: Commit			LocalThreadHijacking.exe (2108) (0x7f6562000 - 0x7f65621000)		
0x7f6563654000	Image: Commit				3616	3616
0x7f6563827000	Image: Commit	000000000	B	48 83 64 20 e9 c0 00 00 00 01 51 41 50 52 51	416	416
0x7f6563827000	Image: Commit	000000010	B	54 48 31 42 45 48 5b 52 60 48 5b 52 18 48 5b	816	816
0x7f6563827000	Image: Commit	000000020	B	48 8b 72 50 0f 87 b7 44 4a 31 c9 49 31 c0	1168	1168
0x7f65637a3000	Image: Commit	000000030	ac	3c 61 7c 02 2c 01 c1 c9 0d 01 01 c2 e2	3216	416
0x7f65636a000	Image: Commit	000000040	52	41 51 49 89 52 08 8b 42 3c 48 01 40	Base of	2048
0x7f656292000	Image: Commit	000000050	00	00 48 85 01 00 48 85 01 00 50 8b 18	1216	1216
0x7f656465000	Image: Commit	000000060	B	48 49 51 02 48 53 e6 47 ff c9 41 38 48 81	1416	1416
0x7f656533000	Image: Commit	000000070	01	d6 44 31 c9 43 31 c0 ac c1 c9 0d 01	1616	1616
0x7f656468000	Image: Commit	000000080	68	75 21 41 64 24 04 48 45 39 41 67 58	1816	1816

Cette technique nous permet de ne pas utiliser les fonctions **VirtualAlloc/Ex** et **VirtualProtect/Ex**.

Le programme peut accéder aux données comme si elles étaient déjà en mémoire, sans utiliser fread. (thread?)

L'OS charge les données du fichier seulement quand elles sont utilisées (chargement à la demande).

Et donc notre attaque consiste à partager notre ressource (payload/shellcode) avec un autre processus :

La **mapping injection** consiste à utiliser une **zone de mémoire mappée (shared section)** pour partager des données entre deux processus.

Le code ou les données sont écrits dans la mémoire mappée depuis un processus, puis deviennent **visibles dans l'autre processus qui mappe la même section**. Cela permet de faire exécuter du code dans un autre processus **sans utiliser directement WriteProcessMemory**.

Donc ca nous donne pour la local mapping injection :

CreateFileMapping

```
HANDLE CreateFileMappingA(  
[in] HANDLE hFile,   
[in, optional] LPSECURITY_ATTRIBUTES lpFileMappingAttributes, // Not  
Required - NULL  
[in] DWORD flProtect,   
[in] DWORD dwMaximumSizeHigh, // Not Required - NULL  
[in] DWORD dwMaximumSizeLow,   
[in, optional] LPCSTR lpName // Not Required - NULL  
);
```

The 3 required parameters for this technique are explained below. The parameters marked as not required can be set to `NULL`.

- **hFile** - A handle to a file from which to create a file mapping handle. Since creating file mapping from a file is not required in the implementation, the **INVALID_HANDLE_VALUE** flag can be used instead. The **INVALID_HANDLE_VALUE** flag is explained by Microsoft:

If `hFile` is `INVALID_HANDLE_VALUE`, the calling process must also specify a size for the file mapping object in the `dwMaximumSizeHigh` and `dwMaximumSizeLow` parameters. In this scenario, `CreateFileMapping` creates a file mapping object of a specified size that is backed by the system paging file instead of by a file in the file system.

Setting this flag allows the function to perform its task without using a file from disk, and instead the file mapping object is created in memory with a size specified by the `dwMaximumSizeHigh` or `dwMaximumSizeLow` parameters.

- **flProtect** - Specifies the page protection of the file mapping object. In this implementation, it will be set as **PAGE_EXECUTE_READWRITE**. Note that this does not create an **RWX** section, but instead it specifies that it can be created later on. If it had been set to **PAGE_READWRITE**, then it would not be possible to execute the payload later on.
- **dwMaximumSizeLow** - The size of the file mapping handle returned. The value of this will be the payload's size.

MapViewOfFile

```
LPVOID MapViewOfFile(  
[in] HANDLE    hFileMappingObject,  
[in] DWORD     dwDesiredAccess,  
[in] DWORD     dwFileOffsetHigh,           // Not Required - NULL  
[in] DWORD     dwFileOffsetLow,           // Not Required - NULL  
[in] SIZE_T     dwNumberOfBytesToMap  
):
```

The 3 required parameters for this technique are explained below. The parameters marked as not required can be set to `NULL`.

- `hFileMappingObject` - The returned handle from the `CreateFileMapping` WinAPI, which is the file mapping object.
- `dwDesiredAccess` - The type of access to a file mapping object, which determines the page protection of the page created. In other words, the memory permissions of the allocated memory by the `MapViewOfFile` call. Since `CreateFileMapping` was set to `PAGE_EXECUTE_READWRITE`, this parameter will use both the `FILE_MAP_EXECUTE` and `FILE_MAP_WRITE` flags to return valid executable and writable memory, which is what is needed to copy the payload and execute it after.

Had the `PAGE_READWRITE` flag been used in `CreateFileMapping` and the `FILE_MAP_EXECUTE` flag was used in `MapViewOfFile`, then `MapViewOfFile` would have failed because executable memory was attempted to be made from a readable and writable `CreateFileMapping` object handle which is not possible.

- `dwNumberOfBytesToMap` - The size of the payload.

Et pour unmapper la mémoire mappé on utilise (à utiliser après que l'injection se soit exécuté)

`UnmapViewOfFile`